

“Property Finder”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

Yashwanth :2520030259

Bharath :2520030169

Praneeth :2520030186

Under the Guidance of

Dr. V.Sireesha

Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled “Property Finder” is a record of Bonafide work of Yashwanth - 2520030259, Bharath - 2520030169, Praneeth - 2520030186 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

Yashwanth - 2520030259

Bharath - 2520030169

Praneeth - 2520030186

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is certify that the mini project based report entitled “Property Finder” is a bonafide work done and submitted by Yashwanth - 2520030259, Bharath - 2520030169, Praneeth - 2520030186 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, during the academic year 2026-27.

Signature of the Guide
(Dr. V.Sireesha)

Signature of the HOD

ACKNOWLEDGEMENT

We express our sincere gratitude to Dr. V.Sireesha, Professor, Department of Computer Science and Engineering, for her guidance, constructive suggestions, and continuous encouragement throughout the development of the Property Finder project. Her feedback helped us refine the problem definition, algorithmic design, implementation strategy, and testing process.

We also thank Dr. P. Venkateswara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the academic support and departmental environment required to complete the work. We are grateful to our faculty members, classmates, and everyone who contributed feedback during demonstrations and evaluation. Finally, we thank our team members for the collaboration, testing, debugging, and documentation efforts that brought the system to its completed phase.

Yashwanth - 2520030259

Bharath - 2520030169

Praneeth - 2520030186

Abstract

Property Finder is a Java-based real-estate search and decision-support prototype developed to demonstrate how advanced Data Structures and Algorithms (DSA) can be integrated into a practical property portal. The system focuses on Telangana and models property records with location, district, type, price, bedrooms, area, amenities, and description attributes. Instead of depending on built-in substring operations for the core search logic, the implementation provides multiple manually coded pattern-matching strategies and exposes the result through both a command-line interface and a local web interface.

The current runtime combines Naive String Matching, Knuth-Morris-Pratt (KMP), Z Algorithm, Rabin-Karp, and Boyer-Moore for exact matching; Levenshtein distance for typo-tolerant location suggestions; a Trie for prefix auto-completion; Breadth-First Search (BFS) for nearby-district exploration; 0/1 Knapsack dynamic programming for investor portfolio selection; and Randomized Quicksort for price and area ordering. The Java web server exposes endpoints for search, autocomplete, investor optimization, nearby districts, and property retrieval, while the browser layer renders search results and property cards.

Functional validation on the supplied implementation confirms that the runtime dataset contains 46 properties across 20 district labels. Representative tests include typo correction from "Gachiboli" to "Gachibowli", autocomplete for the prefix "Gach", retrieval of properties matching "Gym" and "3 BHK", BFS exploration from Warangal, and a ₹15,000,000 investment optimization that selects two land properties totaling 87,120 square feet. A separate local benchmark on the current 46-property dataset also shows the practical advantage of Boyer-Moore for the tested patterns, although performance remains dependent on input size and runtime conditions.

Keywords

Property Finder, Data Structures and Algorithms, String Matching, KMP, Boyer-Moore, Levenshtein Distance, Trie, BFS, Dynamic Programming, 0/1 Knapsack, Randomized Quicksort, Java, Telangana.

Table of Contents

1. INTRODUCTION	1
1.1 Overview	1
1.2 Background and Motivation	1
1.3 Problem Statement	2
1.4 Objectives	3
1.5 Scope of the Project	3
1.6 Significance of the Work	3
1.7 Organization of the Report	4
2. SYSTEM REQUIREMENTS	5
2.1 Existing System and Limitations	5
2.2 Proposed and Implemented System	5
2.3 Functional Requirements	7
2.4 Non-Functional Requirements	7
2.5 Hardware and Software Requirements	8
2.6 Dataset Requirements	8
2.7 Tools and Technologies	9
3. SYSTEM DESIGN	10
3.1 High-Level Architecture	10
3.2 Data Model	10
3.3 Module Design	11
3.4 Search and Ranking Design	12
3.5 Fuzzy Matching and Autocomplete Design	12
3.6 Graph-Based Proximity Design	13
3.7 Investor Optimization Design	13
3.8 Sorting Design	13
3.9 Web API and Front-End Design	14
3.10 Data Flow and Control Flow	14
4. IMPLEMENTATION	15
4.1 Project Structure	15

4.2 Property Model and Data Generation	16
4.3 Property Search Service	16
4.4 Exact String Matching Algorithms	16
4.5 Levenshtein Fuzzy Matching	17
4.6 Trie Auto-Completion	18
4.7 BFS Nearby Districts	18
4.8 0/1 Knapsack Investor Mode	18
4.9 Randomized Quicksort	18
4.10 Local Web Server and API	19
4.11 Front-End Implementation	19
4.12 CO-Wise Implementations	19
4.13 Pseudocode Summary	20
5. RESULTS	24
5.1 Experimental Setup	24
5.2 Dataset Profile	24
5.3 Functional Validation	24
5.4 Search Performance Benchmark	26
5.5 Investor Optimization Result	27
5.6 Graph Proximity Result	27
5.7 Discussion of Results	28
6. CHALLENGES & LIMITATIONS	29
6.1 Issues Faced During Development	29
6.2 Solutions Applied	30
6.3 Limitations of the System	30
7. CONCLUSION & FUTURE WORK	31
7.1 Conclusion	31
7.2 Future Work	31
7.3 Skills Learned During Development	32
REFERENCES	33

List of Figures

Fig. 3.1 System Architecture of Property Finder	10
Fig. 3.2 Property Search Workflow	12
Fig. 3.3 Trie-Based Location Auto-Completion	13
Fig. 3.4 Simplified District Graph and BFS Proximity Search	13
Fig. 3.5 0/1 Knapsack Investor Mode - Design	13
Fig. 3.6 Randomized Quicksort for Property Ranking	14
Fig. 3.7 Web Interface and Local API Interaction	14
Fig. 4.1 Sample Property Imagery Used by the Web Cards	16
Fig. 5.1 Runtime Property Distribution by Type	24
Fig. 5.2 Reproduced Local Benchmark - Average Search Time	26

List of Tables

Table 2.1	Functional Requirements	6
Table 2.2	Non-Functional Requirements	6
Table 2.3	Hardware and Software Requirements	8
Table 3.1	Core Data Model Fields	10
Table 3.2	Module-to-Algorithm Mapping	11
Table 4.1	Source Files and Responsibilities	15
Table 4.2	CO-Wise Implementation Mapping	19
Table 5.1	Functional Validation Results	24
Table 5.2	Pattern-Matching Complexity Comparison	26
Table 5.3	Local Benchmark Results	27

1. INTRODUCTION

1.1 Overview

Property Finder is an academic real-estate search and optimization prototype implemented in Java. The project is intentionally designed around Data Structures and Algorithms rather than a database-heavy commercial stack. Its purpose is to demonstrate how fundamental and advanced algorithmic techniques can be combined into one usable system that supports property discovery, typo tolerance, location suggestions, proximity exploration, investment optimization, and result sorting.

The runtime implementation models property records with a compact Property class and generates a Telangana-focused dataset through TelanganaPropertyData. The current source creates 46 property entries distributed across locations in Hyderabad, Warangal, Karimnagar, Nizamabad, Khammam, Nalgonda, Bhadrachalam, Mahabubabad, Jagtial, Siddipet, Sangareddy, Medak, Suryapet, Adilabad, Mahbubnagar, Vikarabad, Mancherla, Peddapalli, Wanaparthy, and Ranga Reddy. This regional focus keeps the demonstration concrete while still giving enough variation to exercise the algorithms.

The application can be used from the command line or through a browser. The command-line application presents an interactive menu, while the local web server exposes JSON endpoints through Java HttpServer. In both interfaces, the same core services and algorithm implementations are reused, keeping the system modular and easy to demonstrate.

1.2 Background and Motivation

Property search is an information-retrieval problem. A user does not normally know the internal structure of a listing collection; instead, the user expresses a need using words such as a locality name, property type, amenity, or room configuration. The project therefore uses string matching as a central operation. A scalable design should also reduce the friction caused by spelling mistakes and should provide suggestions while the query is being typed.

The motivation for the project is also educational. A typical application can hide algorithmic work behind database indexes or framework utilities, whereas a DSA project is expected to make the computational choices explicit. Property Finder therefore implements multiple exact pattern-matching algorithms, fuzzy matching through dynamic programming, a Trie for prefix search, graph traversal for district relationships, 0/1 Knapsack for budget-constrained portfolio selection, and randomized Quicksort for ranking.

Relationship to the supplied project proposal

The supplied project presentation proposed a broader set of real-estate functions, including add/update/delete listings, multi-criteria filtering, favorites, comparisons, nearby facilities, and administration. The checked-in implementation delivers a narrower but coherent DSA-focused scope: property search, autocomplete, fuzzy location suggestions, nearby-district traversal, investor optimization, all-property retrieval, sorting, a web interface, and standalone CO-wise demonstrations. The additional proposal items are therefore treated as future extensions rather than being reported as implemented features.

1.3 Problem Statement

A property portal may contain many heterogeneous records. A user needs a mechanism to locate relevant records quickly without manually scanning every listing. Simple sequential string checks can become expensive when descriptions grow, and exact matching alone does not address typing errors such as "Gachiboli" instead of "Gachibowli". A second decision problem appears for investors: when the available budget is limited, the user may want a combination of properties that maximizes total area without exceeding that budget. A third challenge is representing location relationships so that nearby districts can be discovered through graph traversal.

The problem addressed by this project is therefore: design and implement a modular Property Finder system in Java that demonstrates efficient searching, typo-tolerant location assistance, prefix autocomplete, graph-based proximity lookup, budget-constrained portfolio optimization, and randomized sorting using explicit DSA

implementations.

1.4 Objectives

- Build a reusable Java property model and Telangana-oriented runtime dataset.
- Implement and integrate five exact pattern-matching algorithms: Naive, KMP, Z, Rabin-Karp, and Boyer-Moore.
- Provide typo-tolerant location suggestions using Levenshtein edit distance.
- Provide prefix autocomplete using a Trie data structure.
- Model selected Telangana district relationships as a graph and perform BFS up to two levels.
- Optimize investor choices with a 0/1 Knapsack dynamic-programming model that maximizes area within a budget.
- Rank search results or the full property list using randomized Quicksort by price or area.
- Expose the core functionality through both a CLI and a local browser-based interface.
- Demonstrate course outcomes CO1 to CO6 through standalone Java programs mapped to the project concepts.

1.5 Scope of the Project

The scope includes data generation, algorithm implementation, service-layer orchestration, a local HTTP server, a responsive browser interface, and functional benchmarking. The current project does not use a persistent relational or NoSQL database. Property records are created in Java at runtime, while a smaller CSV file is also included as project data. The system is intended for educational demonstration and algorithmic evaluation rather than production real-estate transactions.

1.6 Significance of the Work

The main significance is the integration of multiple DSA techniques into a single domain application. Instead of studying KMP, Trie, BFS, Knapsack, and Quicksort as isolated exercises, the project shows where each technique fits naturally into a real application workflow. The same property records can be searched using different

algorithms, explored geographically through graph traversal, optimized under a budget, and sorted for presentation.

The project also illustrates an important software-engineering principle: the user interface can remain relatively simple when the domain services are well separated. `PropertySearchService` handles search and suggestion logic; `InvestorService` handles optimization; algorithm classes encapsulate computational strategies; and `WebServer` exposes the functionality through small API handlers.

1.7 Organization of the Report

Chapter 1 introduces the project, problem, objectives, scope, and motivation. Chapter 2 states functional, non-functional, hardware, software, dataset, and technology requirements and summarizes the conceptual gap addressed by the implementation. Chapter 3 presents the system architecture and algorithmic design. Chapter 4 explains the implementation in module-wise form and provides pseudocode for the major algorithms. Chapter 5 reports functional validation, dataset statistics, benchmark results, and representative outputs. Chapter 6 documents development challenges and system limitations. Chapter 7 concludes the work and identifies future enhancements and skills learned. References are listed at the end.

2. SYSTEM REQUIREMENTS

2.1 Existing System and Limitations

Conventional property-search systems typically depend on database queries, search indexes, filters, and application-level ranking. Such systems are effective for production-scale retrieval, but they often conceal the underlying algorithmic behavior from a DSA learner. For an academic implementation, that creates a gap between the theory of pattern matching or optimization and the software that students can directly inspect.

The project materials identify four recurring needs: fast lookup in text-heavy property records, error tolerance for misspelled locations, optimization of property choices under budget constraints, and efficient traversal of location relationships. A single basic search loop does not address all four needs. Property Finder addresses the gap by assigning an explicit data structure or algorithm to each need.

Limitations of a basic linear-search approach

- Repeated character comparisons can become expensive for long texts and patterns.
- A basic exact search cannot recover gracefully from user spelling errors.
- Prefix suggestions require a structure that can retrieve words by character prefix rather than by full comparison.
- Budget optimization involves a combinatorial choice among properties, which is naturally modeled with dynamic programming.
- Proximity queries are relationship queries, which are more naturally represented as graph traversal.

2.2 Proposed and Implemented System

The proposed system is a layered Property Finder prototype. The presentation layer contains HTML, CSS, and JavaScript. The application layer is a Java `HttpServer` with small REST-style GET endpoints. Service classes implement business logic, algorithm classes perform the computational work, and data/model classes provide the runtime property objects. A command-line `Main` class offers the same major functionality for

direct algorithmic demonstration.

Table 2.1 Functional Requirements

ID	Requirement	Implemented Behavior
FR-01	Property search	Search title, location, district, type, amenities, and description using the selected matcher.
FR-02	Fuzzy location suggestion	Suggest the closest known location when Levenshtein similarity crosses the project threshold.
FR-03	Autocomplete	Return location or district completions for a typed prefix through a Trie.
FR-04	Nearby districts	Traverse the simplified district graph with BFS up to two levels.
FR-05	Investor optimization	Select a portfolio maximizing area within the given budget using 0/1 Knapsack.
FR-06	Sorting	Sort properties by price ascending or area descending using randomized Quicksort.
FR-07	Web API	Expose search, autocomplete, investor, nearby, and property-listing endpoints.
FR-08	CLI operation	Provide an interactive terminal menu for direct demonstration of the features.

Table 2.2 Non-Functional Requirements

Category	Requirement
Usability	Search, investor, nearby, and all-properties functions should be simple to demonstrate from a browser or terminal.
Modularity	Algorithm classes should be independently reusable and services should orchestrate them without duplicating logic.
Performance	Search strategies should exhibit their expected complexity and Boyer-Moore should benefit from character skipping on suitable inputs.
Portability	The project should run with standard Java tooling through run.bat or run.sh scripts.
Maintainability	Classes are separated by model, service, algorithm, data, utility, and web responsibilities.
Transparency	The system should expose which algorithm matched a result in the search workflow.

2.3 Functional Requirements

The functional requirements are centered on the DSA demonstrations. Search must operate across multiple property fields, rather than only on the title. Autocomplete and typo correction are treated as separate assistance mechanisms. Investor mode must return the selected properties and aggregate cost and area. Nearby mode must return district names reached by BFS. Sorting must preserve property objects while changing their order for presentation.

2.4 Non-Functional Requirements

The application is designed for clarity, repeatability, and ease of classroom demonstration. It avoids unnecessary infrastructure, runs from local scripts, and keeps the algorithm implementations visible in source form. Because the application is an

academic prototype, correctness and interpretability are emphasized alongside performance. Benchmark timings are treated as environment-specific observations rather than universal performance guarantees.

2.5 Hardware and Software Requirements

Table 2.3 Hardware and Software Requirements

Type	Requirement	Purpose
Hardware	Laptop/desktop with standard keyboard and display	Development, compilation, and browser testing.
Operating System	Windows or Linux/macOS terminal environment	Run the supplied .bat or .sh scripts.
JDK	Java Development Kit; project benchmark materials use Java 17	Compile and execute the Java source.
Browser	Modern Chromium/Chrome/Firefox/Edge browser	Open the local web interface.
Editor/IDE	Any Java-capable IDE or text editor	Modify source files and inspect the implementation.
Storage	Local filesystem access	Store source code, images, scripts, and report artifacts.

2.6 Dataset Requirements

The core runtime data is generated by `TelanganaPropertyData.generateData()`. Each `Property` object contains an identifier, title, location, district, property type, price, bedroom count, area in square feet, amenities, and description. The checked-in source currently generates 46 records. The separate `data/properties.csv` file contains a smaller sample dataset and is not the source used by `PropertySearchService` at runtime.

The dataset needs enough textual diversity to exercise multiple matchers. It includes

locality names, district labels, property-type terms, amenities, and descriptions. It also includes large-area land and farmhouse records, making the Knapsack optimization visibly different from apartment-only examples.

2.7 Tools and Technologies

- Java for core implementation and standalone CO-wise programs.
- Standard Java collections such as ArrayList, HashMap, HashSet, Queue, and maps used inside algorithm support structures.
- Java HttpServer for the local API layer.
- HTML, CSS, and JavaScript for the browser interface.
- Lucide icons and remote web fonts referenced by the supplied front-end.
- Local scripts run.bat and run.sh for compilation and execution.

3. SYSTEM DESIGN

3.1 High-Level Architecture

The system follows a lightweight layered architecture. User requests enter through either the CLI or the browser. The browser communicates with the local Java web server using small GET requests. Web handlers translate HTTP parameters into calls to PropertySearchService, InvestorService, or LocationGraph, then JsonUtil serializes the results. The same algorithms are also directly accessible from the CLI path.

Fig. 3.1 System Architecture of Property Finder

3.2 Data Model

The Property class is the principal domain object. It keeps the data needed by every major feature in one record, allowing search, optimization, and sorting to operate on the same in-memory object. This avoids converting between multiple representations during the application flow.

Table 3.1 Core Data Model Fields

Field	Type	Meaning	Used By
propertyId	String	Unique listing identifier such as HYD001.	Search, display, deduplication
title	String	Human-readable property title.	Pattern matching, display
location	String	Locality or neighborhood.	Pattern matching, Trie, fuzzy matching
district	String	District label.	Pattern matching, Trie, BFS context
propertyType	String	Apartment, Villa, House, Land,	Pattern matching,

Field	Type	Meaning	Used By
		Commercial, Farmhouse.	filtering concept
price	double	Property price in INR.	Search output, sorting, Knapsack
bedrooms	int	Number of bedrooms.	Search output, UI display
area	int	Area in square feet.	Sorting, Knapsack
amenities	String	Comma-separated facilities.	Pattern matching, UI display
description	String	Short textual description.	Pattern matching

3.3 Module Design

Table 3.2 Module-to-Algorithm Mapping

Module	Primary Structure / Algorithm	Reason for Use
Property Search	Naive, KMP, Z, Rabin-Karp, Boyer-Moore	Compare alternative exact string-search strategies.
Fuzzy Location	Levenshtein DP	Handle misspellings through edit distance.
Autocomplete	Trie	Retrieve complete locations from prefixes.
Nearby Districts	Graph + BFS	Explore district relationships up to depth two.

Module	Primary Structure / Algorithm	Reason for Use
Investor Mode	0/1 Knapsack DP	Maximize square footage within a budget.
Sorting	Randomized Quicksort	Efficient ranking by price or area.
Web Layer	HttpServer + JSON	Expose services to the browser UI.

3.4 Search and Ranking Design

PropertySearchService loops over every Property record and invokes the selected exact matcher against six fields: title, location, district, property type, amenities, and description. Each property is added to the result list at most once, even if the pattern is found in multiple fields. This deduplication rule ensures that the UI displays a property as a single card instead of repeating the same record for several matching fields.

When results are available, the CLI optionally passes the matched property objects to QuickSortProperties. The browser interface can also request all properties from /api/properties with a sorting option. The search-card UI records the matched field and algorithm name, making the computational path visible to the user.

Fig. 3.2 Property Search Workflow

3.5 Fuzzy Matching and Autocomplete Design

The fuzzy-matching component builds a list of unique locations and district labels, computes Levenshtein similarity against the user input, and returns the highest-scoring candidate when the similarity is at least 40 percent and less than 100 percent. In the supplied demonstration, "Gachiboli" is one edit away from "Gachibowli", producing a very strong correction candidate.

Autocomplete is different from fuzzy matching. A Trie stores characters as a prefix

tree. As the user types a prefix such as "Gach", the Trie follows one child per character and then performs a depth-first traversal of the matching node to collect complete words. The front-end calls /api/autocomplete as the user types and displays the resulting suggestions in a dropdown.

Fig. 3.3 Trie-Based Location Auto-Completion

3.6 Graph-Based Proximity Design

The LocationGraph class uses an adjacency-list representation. Edges model a simplified neighborhood relationship among selected Telangana districts. The BFS routine starts from the requested district, stores discovered vertices in a queue, maintains a visited set, and limits traversal to two levels. This turns a location relationship problem into a standard graph traversal exercise.

Fig. 3.4 Simplified District Graph and BFS Proximity Search

3.7 Investor Optimization Design

Investor mode treats each property as an item whose weight is its price and whose value is its area. Properties above the budget are filtered out first. Because prices are large INR amounts, the implementation scales price and budget by 100,000 before building the DP table. The resulting table stores the maximum total area obtainable for every prefix of items and every scaled budget. A backtracking pass recovers the selected properties.

Fig. 3.5 0/1 Knapsack Investor Mode - Design

3.8 Sorting Design

QuickSortProperties implements randomized Quicksort and supports two ranking criteria: price ascending and area descending. A random pivot is selected, moved to the end, and used for partitioning. The implementation then recursively sorts the left and right sublists. Random pivot selection is used to reduce sensitivity to unfavorable input

order and to retain expected $O(n \log n)$ performance.

Fig. 3.6 Randomized Quicksort for Property Ranking

3.9 Web API and Front-End Design

The local WebServer exposes five endpoints: `/api/search`, `/api/autocomplete`, `/api/investor`, `/api/nearby`, and `/api/properties`. The browser code uses `fetch()` to request JSON, converts responses into interactive property cards or badges, and handles loading states. The front-end provides four tabs: Search, Investor Mode, Nearby, and All Properties.

Fig. 3.7 Web Interface and Local API Interaction

3.10 Data Flow and Control Flow

For a search request, control flows from input to optional autocomplete, to fuzzy suggestion logic, to the selected exact matcher, to property deduplication, and finally to sorting and display. For an investor request, the flow is input budget, affordability filter, scaled DP construction, backtracking, and summary display. For nearby districts, the flow is input district, graph lookup, BFS, and result serialization. These flows share the same property and service layers, which keeps the architecture compact.

4. IMPLEMENTATION

4.1 Project Structure

The supplied repository separates core application code, CO-wise demonstrations, data, reports, results, and browser assets. The separation reflects the educational structure of the project and also makes individual algorithms easy to inspect or execute in isolation.

Table 4.1 Source Files and Responsibilities

Path	Responsibility
src/Main.java	CLI entry point and interactive menu.
src/model/Property.java	Property domain model and getters.
src/data/TelanganaPropertyData.java	Generates the runtime Telangana property dataset.
src/service/PropertySearchService.java	Coordinates search, Trie construction, and fuzzy suggestions.
src/service/InvestorService.java	Performs budget-based 0/1 Knapsack optimization.
src/algorithms/*.java	Manual implementations of matching, Trie, graph, and sorting algorithms.
src/web/WebServer.java	Runs the local Java HTTP server and API handlers.
src/utlis/JsonUtil.java	Converts objects to JSON for the web layer.
public/index.html	Browser structure and tab sections.
public/app.js	Front-end events, API requests, and card rendering.

Path	Responsibility
public/style.css	Responsive glassmorphism-style user interface.

4.2 Property Model and Data Generation

The `Property` constructor centralizes all fields needed for the application. `TelanganaPropertyData.generateData()` instantiates the records by district group. The source contains 46 property declarations, including apartments, houses, land, villas, commercial spaces, and farmhouses. This design makes the demonstration deterministic: the same records are available every time the application is started.

Fig. 4.1 Sample Property Imagery Used by the Web Cards

4.3 Property Search Service

`PropertySearchService` constructs the runtime dataset and initializes a Trie. Each property location and district is inserted into the Trie. The `search()` method receives a pattern and algorithm choice, normalizes the pattern to lowercase, and then applies the selected matcher to the six supported text fields. `SearchResult` records the property, matched field, match position, and algorithm name. A property is added only on the first matching field to prevent duplicates.

Search algorithm selection

The CLI lets the user choose one of five exact matchers. The browser defaults to Boyer-Moore but allows the user to select a different matcher from the algorithm dropdown. The separate CO1 demonstration contains a lightweight algorithm-selector example based on text length, illustrating how different algorithms can be chosen for different input sizes.

4.4 Exact String Matching Algorithms

Naive String Matching

The Naive matcher examines every possible alignment of the pattern against the text. For each alignment it compares characters from left to right. It requires no preprocessing and uses $O(1)$ extra algorithmic space beyond the result list, but its worst-case time is $O(nm)$. It is useful as a baseline and is especially transparent for teaching.

Knuth-Morris-Pratt (KMP)

KMP preprocesses the pattern into an LPS (Longest Prefix Suffix) array. When a mismatch occurs after a partial match, the LPS value determines how far the pattern can fall back without rechecking characters that are already known. The project implementation therefore achieves $O(n + m)$ matching time with $O(m)$ pattern-preprocessing space.

Z Algorithm

The Z matcher forms pattern + delimiter + text and computes a Z-array. A Z value equal to the pattern length indicates a match. This gives linear $O(n + m)$ time over the concatenated string and uses $O(n + m)$ auxiliary space for the Z-array.

Rabin-Karp

Rabin-Karp compares a rolling hash for the pattern with the hash of each text window. When the hash values match, the code performs character-by-character verification to handle collisions. The implementation uses a polynomial-style rolling hash with alphabet base 256 and prime modulus 101. Average behavior is good, while the worst case remains $O(nm)$ due to collisions and verification.

Boyer-Moore

Boyer-Moore scans the pattern from right to left and uses a bad-character table to skip positions after a mismatch. This can significantly reduce the number of comparisons on suitable text and patterns. The supplied benchmark report and the reproduced local benchmark both show Boyer-Moore as the fastest among the tested choices for their respective setups, although exact timing depends on input and environment.

4.5 Levenshtein Fuzzy Matching

Levenshtein distance measures the minimum number of single-character insertions, deletions, and substitutions needed to transform one string into another. The implementation uses a two-dimensional dynamic-programming table. Similarity is calculated as $(1 - \text{distance}/\text{maxLength}) \times 100$. PropertySearchService compares the user input with known locations and returns the best candidate when the similarity is at least 40 percent and below 100 percent.

4.6 Trie Auto-Completion

The TrieAutocomplete class stores characters in a tree of TrieNode objects. Each node maintains a map of child characters and an end-of-word flag. Insert operations convert words to lowercase and create missing nodes. Autocomplete first traverses the prefix and then recursively collects every complete word below the matched node. The front-end uses this feature continuously after at least two input characters have been typed.

4.7 BFS Nearby Districts

LocationGraph stores an adjacency list in a HashMap. The getNearbyDistricts method initializes a queue with the requested district and a visited set. It processes the graph level by level and stops after two BFS layers, returning the discovered districts in traversal order. The graph itself is described in the source as a simplified mock graph for the project demonstration rather than a complete geographic model.

4.8 0/1 Knapsack Investor Mode

InvestorService filters out properties whose individual price exceeds the supplied budget. Prices are scaled by 100,000 to keep the two-dimensional DP array manageable. The recurrence is the standard 0/1 Knapsack relation: if an item fits, the algorithm compares including the property with excluding it; otherwise it carries forward the exclusion value. Backtracking reconstructs the selected property set and computes total cost and total area.

4.9 Randomized Quicksort

QuickSortProperties exposes two public methods: sortByPriceAsc and sortByAreaDesc. Both call the same randomized partitioning routine with a boolean

determining the comparison direction. The random pivot is swapped to the high index before partitioning. Recursive partitioning continues until the subarray contains fewer than two items.

4.10 Local Web Server and API

WebServer uses Java HttpServer and binds to port 8081. Each API handler extracts query parameters, invokes a service, and serializes the result. The handlers deliberately remain small, which preserves the separation between transport and business logic. The server sets an application/json content type for responses and adds permissive CORS headers for local browser requests.

Implemented endpoints

- /api/search?pattern=...&algo=... returns a suggestion and search results.
- /api/autocomplete?prefix=... returns matching Trie suggestions.
- /api/investor?budget=... returns the optimized portfolio and aggregate totals.
- /api/nearby?district=... returns up to two levels of reachable districts.
- /api/properties?sort=price|area returns the complete property list, optionally sorted.

4.11 Front-End Implementation

The browser interface is implemented in index.html, style.css, and app.js. The visual layer uses a dark glassmorphism design with four tabs. Property cards are rendered from a reusable HTML template. The app assigns bundled local images to the cards through the project assets and otherwise falls back to the front-end data model. Search input triggers autocomplete after a short debounce period, while investor and nearby actions make asynchronous fetch requests and update the page without reloading.

4.12 CO-Wise Implementations

Table 4.2 CO-Wise Implementation Mapping

CO	Standalone File	Concept Demonstrated	Relationship to Main Application
CO1	CO1/AlgorithmSelector.java	Algorithm selection based on text size.	Supports the rationale for choosing an appropriate exact matcher.
CO2	CO2/StringMatching.java	KMP string matching.	Represents the KMP branch of the main search module.
CO3	CO3/EditDistance.java	Levenshtein edit distance.	Matches the fuzzy location correction mechanism.
CO4	CO4/BipartiteMatching.java	DFS-based maximum bipartite matching.	Provides an additional graph/matching demonstration; the core application uses BFS for district proximity.
CO5	CO5/KnapsackPropertySelection.java	0/1 Knapsack dynamic programming.	Directly matches Investor Mode.
CO6	CO6/RandomizedQuickSort.java	Randomized Quicksort.	Directly matches property ranking by price or area.

4.13 Pseudocode Summary

Algorithm 1: Property Search Orchestration

INPUT: pattern, algorithmChoice

LOAD all properties

```

FOR each property p
  FOR each supported text field f in p
    matches ← MATCH(f, pattern, algorithmChoice)
    IF matches is non-empty AND p not already in results THEN
      ADD p and match metadata to results
      BREAK inner field loop
    END IF
  END FOR
END FOR
RETURN results

```

Algorithm 2: Naive Pattern Matching

```

FOR i ← 0 TO n-m
  j ← 0
  WHILE j < m AND text[i+j] = pattern[j]
    j ← j + 1
  IF j = m THEN record i as a match
END FOR

```

Algorithm 3: KMP Pattern Matching

```

BUILD LPS array for pattern
i ← 0, j ← 0
WHILE i < n
  IF text[i] = pattern[j] THEN increment i and j
  IF j = m THEN record i-j; set j ← LPS[j-1]
  ELSE IF mismatch and j > 0 THEN j ← LPS[j-1]
  ELSE increment i
END WHILE

```

Algorithm 4: Z Pattern Matching

```

CONCATENATE pattern + delimiter + text
COMPUTE Z array using [L,R] window maintenance
FOR each Z[i]
  IF Z[i] = pattern length THEN record translated text position
RETURN all recorded positions

```

Algorithm 5: Rabin-Karp

```
COMPUTE hash(pattern) and hash(first text window)
FOR each window position i
    IF hashes are equal THEN verify all characters
    IF verification succeeds THEN record i
    UPDATE the text-window hash using rolling arithmetic
RETURN all positions
```

Algorithm 6: Boyer-Moore (Bad Character Heuristic)

```
BUILD bad-character last-occurrence table
s ← 0
WHILE s ≤ n-m
    COMPARE pattern from right to left
    IF full match THEN record s and shift
    ELSE SHIFT using the bad-character rule
END WHILE
```

Algorithm 7: Levenshtein Distance

```
CREATE DP table of size  $(len1+1) \times (len2+1)$ 
INITIALIZE first row and first column with edit counts
FOR i from 1..len1
    FOR j from 1..len2
        cost ← 0 if characters equal else 1
         $dp[i][j] \leftarrow \min(\text{delete, insert, substitute})$ 
RETURN  $dp[len1][len2]$ 
```

Algorithm 8: Trie Auto-Complete

```
INSERT each location and district into the Trie
FOR a query prefix, follow one child node per character
IF any character path is missing THEN return empty
OTHERWISE DFS from the prefix node and collect end-of-word nodes
RETURN collected suggestions
```

Algorithm 9: BFS Nearby Districts

```
CHECK that start district exists in adjacency list
```

```

QUEUE start district; mark it visited
FOR up to two BFS levels
    PROCESS every node at the current level
    FOR each unvisited neighbor
        mark visited; enqueue; add to result
RETURN discovered districts

```

Algorithm 10: 0/1 Knapsack Investor Optimization

```

FILTER properties with price  $\leq$  budget and area  $> 0$ 
SCALE prices and budget by 100,000
CREATE DP table  $K[i][w]$ 
FOR each property  $i$  and capacity  $w$ 
    IF item fits:  $K[i][w] \leftarrow \max(\text{value} + K[i-1][w-\text{weight}], K[i-1][w])$ 
    ELSE  $K[i][w] \leftarrow K[i-1][w]$ 
BACKTRACK from  $K[n][W]$  to recover selected properties
RETURN selected properties, total cost, and total area

```

Algorithm 11: Randomized Quicksort

```

IF low < high THEN
    CHOOSE a random pivot index
    SWAP pivot with high
    PARTITION list around pivot
    RECURSE on left and right sublists
END IF

```

5. RESULTS

5.1 Experimental Setup

The results chapter combines three sources of evidence from the supplied project: (1) direct execution of the current source code, (2) the project-provided sample output and benchmark report, and (3) a reproduced local benchmark run performed on the current 46-property runtime dataset. The reproduced benchmark used the concatenated property text generated by the current PropertySearchService and repeated each matcher 5,000 times after 2,000 warm-up calls per algorithm.

Because Java execution time depends on processor load, JVM state, garbage collection, and input characteristics, the timings should be interpreted comparatively rather than as fixed guarantees. The benchmark is therefore useful for observing relative behavior, while functional outputs are used to validate correctness.

5.2 Dataset Profile

The current runtime dataset contains 46 properties. The type distribution is dominated by apartments, followed by houses and land. The source also includes four villas, four commercial properties, and two farmhouses. The dataset spans 20 district labels, with Hyderabad contributing the largest group of ten properties.

Fig. 5.1 Runtime Property Distribution by Type

The minimum listed price in the runtime dataset is ₹3,000,000 and the maximum is ₹45,000,000. Area ranges from 500 square feet to 217,800 square feet, with large agricultural/farmhouse records contributing strongly to the upper end. The average values are approximately ₹10.45 million per property and 10,707 square feet of area. These statistics indicate a deliberately mixed dataset rather than a uniform set of apartment listings.

5.3 Functional Validation

Table 5.1 Functional Validation Results

Test Case	Input	Expected / Observed Result	Status
TC-01	Search: Gachibowli, KMP	1 matching runtime property; HYD001 is returned.	PASS
TC-02	Search: Gym	4 matching runtime properties across text fields.	PASS
TC-03	Search: 3 BHK	5 matching runtime properties.	PASS
TC-04	Fuzzy location: Gachiboli	Suggests Gachibowli; edit distance = 1 in the standalone demonstration.	PASS
TC-05	Autocomplete prefix: Gach	Returns Gachibowli from the Trie.	PASS
TC-06	Investor budget: ₹15,000,000	Selects Ananthagiri farm land + Geesugonda agricultural land; total cost ₹15M, area 87,120 sqft.	PASS
TC-07	Nearby district: Warangal	Returns Hanamkonda, Karimnagar, Mahabubabad and second-level reachable districts.	PASS
TC-08	Sort by price ascending	Properties are ordered from lower to higher price by randomized Quicksort.	PASS

Test Case	Input	Expected / Observed Result	Status
TC-09	Sort by area descending	Properties are ordered from larger to smaller area.	PASS
TC-10	Web API: /api/properties	Returns the JSON list of runtime properties.	PASS

5.4 Search Performance Benchmark

Table 5.2 Pattern-Matching Complexity Comparison

Algorithm	Best / Average	Worst Case	Extra Space
Naive	$O(n)$ to $O(nm)$	$O(nm)$	$O(1)$
KMP	$O(n + m)$	$O(n + m)$	$O(m)$
Z Algorithm	$O(n + m)$	$O(n + m)$	$O(n + m)$
Rabin-Karp	$O(n + m)$ average	$O(nm)$	$O(1)$ in matcher core
Boyer-Moore	Often sublinear in practice	$O(nm)$	$O(m + \Sigma)$
Levenshtein	$O(\text{len1} \cdot \text{len2})$	$O(\text{len1} \cdot \text{len2})$	$O(\text{len1} \cdot \text{len2})$

The supplied benchmark report describes a separate setup based on 5,000 generated property descriptions under Java 17 and records average search times of 45.2 ms for Naive, 38.5 ms for Rabin-Karp, 12.1 ms for KMP, 14.3 ms for Z Algorithm, and 8.7 ms for Boyer-Moore. Those numbers are retained here as project-reported evidence and are not treated as measurements of the current 46-property dataset.

Fig. 5.2 Reproduced Local Benchmark - Average Search Time

Table 5.3 Local Benchmark Results

Algorithm	Gachibowli (ms)	Security (ms)	3 BHK (ms)	3-Pattern Average (ms)
Naive	0.012386	0.012788	0.013412	0.012862
KMP	0.014669	0.014492	0.014043	0.014401
Z Algorithm	0.013419	0.013847	0.013007	0.013424
Rabin-Karp	0.016601	0.016837	0.017301	0.016913
Boyer-Moore	0.002711	0.004556	0.004294	0.003854

In the reproduced benchmark, Boyer-Moore is fastest for all three tested patterns. The result is consistent with the algorithm’s bad-character skipping strategy, but the small 4,038-character test text also means preprocessing overhead can influence the ranking. The benchmark therefore supports the educational conclusion that algorithm choice can matter, while also demonstrating that asymptotic complexity should not be interpreted without considering input size and implementation overhead.

5.5 Investor Optimization Result

For a budget of ₹15,000,000, the current runtime optimizer selects two land properties: Farm Land in Ananthagiri priced at ₹5,000,000 with 43,560 square feet and Agricultural Land in Geesugonda priced at ₹10,000,000 with 43,560 square feet. The combination uses the budget exactly and produces 87,120 square feet, which is larger than the area of any single affordable property in the dataset.

5.6 Graph Proximity Result

For the input Warangal, the current BFS implementation returns Hanamkonda, Karimnagar, Mahabubabad, and second-level reachable districts including Jagtial, Peddapalli, Siddipet, and Khammam. The result demonstrates level-based graph exploration. The implementation is intentionally simplified and should be interpreted

as a DSA graph model, not as a geographic routing engine.

5.7 Discussion of Results

The functional tests show that the algorithms are correctly connected to the domain features: string matching returns the expected query results, fuzzy matching identifies the intended spelling correction, Trie autocomplete provides prefix suggestions, BFS explores the modeled district graph, Knapsack returns a budget-feasible portfolio, and Quicksort changes ordering without changing the underlying property records. The benchmark also distinguishes theoretical from applied performance—KMP and Z provide linear guarantees, while Boyer-Moore can be sublinear in practice; the observed ranking remains dependent on input size, pattern characteristics, preprocessing cost, cache effects, and runtime environment.

6. CHALLENGES & LIMITATIONS

6.1 Issues Faced During Development

Handling large budget values in dynamic programming

A direct Knapsack table indexed by raw INR values would require an impractically large second dimension. The implementation addresses this by scaling prices and budget by 100,000. This preserves the intended demonstration while keeping the DP table within manageable memory limits.

Preventing duplicate search results

A property can match the same query in more than one field. Without a guard, the UI would display duplicates. PropertySearchService therefore checks the property identifier before adding a new SearchResult and breaks after the first successful field match.

Providing useful typo correction

Exact matching alone cannot correct user input errors. The fuzzy component computes similarity against a unique location/district list and only proposes a correction when the best score meets the threshold. This keeps the behavior predictable while avoiding unconditional replacements.

Keeping the web layer simple

The project does not rely on a framework-based backend. WebServer implements handlers directly using Java HttpServer. This keeps the implementation transparent, but it also requires explicit query parsing, response creation, and static-file handling.

Reconciling the broad proposal with the implemented scope

The supplied presentation lists a number of desirable commercial features that are not present in the current source. During documentation, these were separated into implemented features versus future extensions so that the report does not overstate the software's capabilities.

6.2 Solutions Applied

- Used budget/price scaling before Knapsack DP table allocation.
- Used a visited set and property-id deduplication in graph and search flows.
- Separated exact matching, fuzzy matching, Trie autocomplete, and ranking into independent classes.
- Used JSON endpoints so the front-end could remain a lightweight static client.
- Added run.bat and run.sh scripts for repeatable compilation and execution.
- Used explicit benchmark disclaimers so timing claims remain tied to the tested environment and input size.

6.3 Limitations of the System

- The runtime dataset is hard-coded in Java and contains 46 properties; it is not a live real-estate inventory.
- The accompanying CSV file is a small sample and is not used by PropertySearchService for runtime loading.
- The district graph is a simplified mock relationship model, not a verified geographic adjacency or routing network.
- The API performs simple query parsing and local JSON responses; it is not designed as a production-grade secure backend.
- The current implementation does not include persistent users, authentication, saved favorites, multi-property comparison, payment, booking, or an admin dashboard.
- The proposed presentation features such as ratings, distance-based sorting, nearby facilities, and CRUD listing management are not implemented in the supplied source.
- Benchmark timings are machine- and input-dependent and should not be interpreted as production-scale performance measurements.
- The web front-end references external fonts and icon resources, so some visual elements may depend on internet availability even though the main data/API path is local.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

Property Finder demonstrates that advanced Data Structures and Algorithms can be integrated into a coherent domain application rather than studied as isolated academic exercises. The completed implementation combines five exact string-matching algorithms, Levenshtein dynamic programming, Trie-based autocomplete, BFS graph traversal, 0/1 Knapsack optimization, and randomized Quicksort around a common Property model and Telangana-focused dataset.

The system successfully connects algorithms to user-facing tasks: matching a location or amenity, assisting with spelling mistakes, suggesting locations while typing, exploring nearby districts, selecting an investment portfolio within a budget, and sorting property records. The dual-interface design, with both CLI and local browser views, makes the project suitable for demonstrations, viva examinations, algorithm analysis, and further experimentation.

Functional validation and benchmarking support the core design. Representative tests produce expected search counts, a Levenshtein correction for "Gachiboli", a valid ₹15 million portfolio with 87,120 square feet, correct two-level BFS traversal from Warangal, and working price/area sorting. The reproduced search benchmark also shows Boyer-Moore performing best among the tested matchers on the current small dataset.

7.2 Future Work

- Replace hard-coded runtime data with a persistent database and authenticated CRUD operations.
- Add multi-criteria filtering for budget, property type, bedrooms, amenities, and district as a dedicated query layer.
- Introduce ranking that combines price, area, user preferences, and distance, potentially using a richer scoring model.
- Replace the simplified district graph with verified geographic relationships and

Dijkstra/A* route support.

- Add favorites, comparison, user accounts, and inquiry workflows.
- Extend nearby search to facilities such as schools, hospitals, metro stations, and transport using a graph-based location model.
- Add a true admin panel for listing moderation and property updates.
- Expand benchmarking to larger synthetic and real-world datasets and compare warm-up, average, percentile, and memory behavior across algorithms.
- Add automated unit tests and integration tests for each endpoint and algorithm implementation.
- Improve web security, validation, error handling, URL decoding, and deployment configuration for a production-quality version.

7.3 Skills Learned During Development

- Manual implementation and comparison of string matching algorithms.
- Dynamic-programming formulation, state design, and backtracking for optimization problems.
- Trie construction, prefix traversal, and recursive collection of suggestions.
- Graph modeling with adjacency lists, BFS queues, and visited-state management.
- Randomized divide-and-conquer sorting and interpretation of expected complexity.
- Java class design, modular services, collections, exception handling, and file/project organization.
- Basic local HTTP server design, JSON serialization, asynchronous browser fetch calls, and UI state updates.
- Benchmark design, result interpretation, and the distinction between theoretical complexity and environment-specific runtime measurements.
- Technical documentation, pseudocode preparation, and mapping course outcomes to an implemented software system.

REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.
- [2] R. Sedgewick and K. Wayne, Algorithms, 4th ed., Addison-Wesley Professional, 2011.
- [3] D. E. Knuth, J. H. Morris, and V. R. Pratt, "Fast pattern matching in strings," SIAM Journal on Computing, vol. 6, no. 2, pp. 323-350, 1977.
- [4] R. M. Karp and M. O. Rabin, "Efficient randomized pattern-matching algorithms," IBM Journal of Research and Development, vol. 31, no. 2, pp. 249-260, 1987.
- [5] R. S. Boyer and J. S. Moore, "A fast string searching algorithm," Communications of the ACM, vol. 20, no. 10, pp. 762-772, 1977.
- [6] Supplied Property Finder README.md and source repository materials, including algorithm implementations, scripts, data, and front-end assets.
- [7] Supplied Property Finder docs/algorithm_analysis.md, used for complexity summaries of the implemented pattern-matching algorithms.
- [8] Supplied Property Finder reports/benchmark_report.md, used as the project-reported benchmark reference for the separate 5,000-description experiment.
- [9] Supplied Property Finder project presentation (property_finder.pptx), used for the original problem framing and proposed functionality list.
- [10] Oracle Java Platform documentation, Java SE APIs and standard collections / HttpServer concepts, consulted as general implementation background.

APPENDIX A - EXECUTION AND SAMPLE COMMANDS

The supplied project includes run.bat for Windows and run.sh for Linux/macOS. The scripts compile the core source and CO-wise demonstrations before presenting a launch menu. The following commands document the intended execution flow.

Windows

```
cd PropertyFinder
```

```
run.bat
```

Choose 1 for the main CLI application or 8 for the web frontend.

Linux / macOS

```
cd PropertyFinder
```

```
chmod +x run.sh
```

```
./run.sh
```

Direct web-server launch

```
cd src
```

```
javac Main.java utils/*.java model/*.java data/*.java algorithms/*.java service/*.java  
web/*.java
```

```
java web.WebServer
```

Open the local interface at <http://localhost:8081/>

Representative CLI interactions

Search example: enter 1, use a pattern such as Gachiboli, accept the Gachibowli suggestion, choose KMP, and optionally sort the results. Autocomplete example: enter 2 and type Gach. Investor example: enter 3 and provide 15000000. Nearby example: enter 4 and provide Warangal. All-property example: enter 5 and select price or area sorting.